

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Расчётно-графическая работа
по дисциплине
«Интернет технологии и сетевое программное обеспечение»

по теме:
Система управления мероприятиями

Студент:
Группа: ИКС - 431

Б.Р Шаимов

Предподаватель:

А.В Лошкарёв

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
О ЧЁМ ПРОЕКТ	4
Архитектура системы	5
Технологический стек и обоснование решений:	6
4. Практическая реализация	7
4.1. Проектирование протокола взаимодействия gRPC.....	7
4.2. Разработка микросервиса Events Service	8
4.3. Разработка API Gateway	9
4.4. Асинхронный сервис уведомлений (Notification Service)	10
5. Инфраструктура и развёртывание (DOCKER).....	11
6. Тестирование программно -аппаратного комплекса.....	11
7. ЗАКЛЮЧЕНИЕ	13

ВВЕДЕНИЕ

В реальной промышленной разработке редко создаются изолированные монолитные серверы; современные программные продукты представляют собой сложные распределенные Системы. Целью данной расчетно-графической работы является проектирование и реализация финального проекта, объединяющего стек технологий: REST, gRPC, Docker и CI/CD. Разработанный аппаратно-программный комплекс демонстрирует навыки принятия технических решений при проектировании распределенных систем. Особое внимание в работе уделено трем ключевым аспектам:

- **Связность:** сервисы спроектированы независимыми друг от друга с использованием брокера сообщений RabbitMQ.
- **Наблюдаемость:** реализовано базовое логирование процессов для отслеживания состояния компонентов.
- **Удобство разработки:** обеспечен запуск всей инфраструктуры одной командой `docker-compose up` без сложной ручной настройки.

О ЧЁМ ПРОЕКТ

Разработка велась на основе индивидуального варианта задания для студента (student_id: s18, группа 431).

Предметная область системы — управление мероприятиями. В рамках модели данных сущность мероприятия обязательно должна содержать текстовое поле location (место проведения).

Для реализации межсервисного взаимодействия заданы следующие сетевые контракты:

- Внешний шлюз (API Gateway) принимает запросы по префиксу /api/events и маршрутизирует их на внутренний сервис.
- Основной сервис (events-svc-s18) обрабатывает запросы на порту 8277.
- Протокол общения реализован через gRPC.

Архитектура системы

Для выполнения задачи была спроектирована система из 3 микросервисов. Использована архитектура, позволяющая комбинировать синхронное и асинхронное взаимодействие.

Обзор компонентов:

API Gateway: Единая точка входа для клиентов. Осуществляет маршрутизацию внешних запросов во внутреннюю сеть.

Events Service: Основной бизнес-сервис для создания и хранения мероприятий.

Notification Service: Асинхронный воркер, отвечающий за обработку событий и отправку уведомлений.

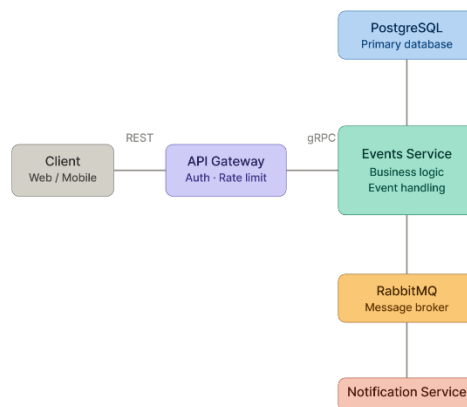


Рисунок 1 - Диаграмма взаимодействия

Технологический стек и обоснование решений:

- Язык программирования: Python (фреймворк FastAPI для шлюза).
- База данных: PostgreSQL в качестве реляционного хранилища.
- Очередь сообщений: RabbitMQ для обеспечения слабой связности системы. В случае падения сервиса уведомлений, создание мероприятий не будет заблокировано.
- Протоколы: Выбран протокол REST для внешнего API, так как он обеспечивает удобную интеграцию с фронтенд-клиентами. Для межсервисного общения между Gateway и Events Service выбран gRPC благодаря высокой скорости сериализации.
- Контейнеризация: Вся инфраструктура упакована с помощью Docker + Compose для консистентного локального запуска.

4. Практическая реализация

4.1. Проектирование протокола взаимодействия gRPC

Основой взаимодействия между внешним шлюзом (API Gateway) и внутренним микросервисом (Events Service) является строгий контракт данных. Для этого был выбран фреймворк gRPC, использующий бинарный протокол сериализации Protobuf. В файле events.proto была описана структура сущности Event с обязательным полем location и RPC-методы для создания и получения списка мероприятий.

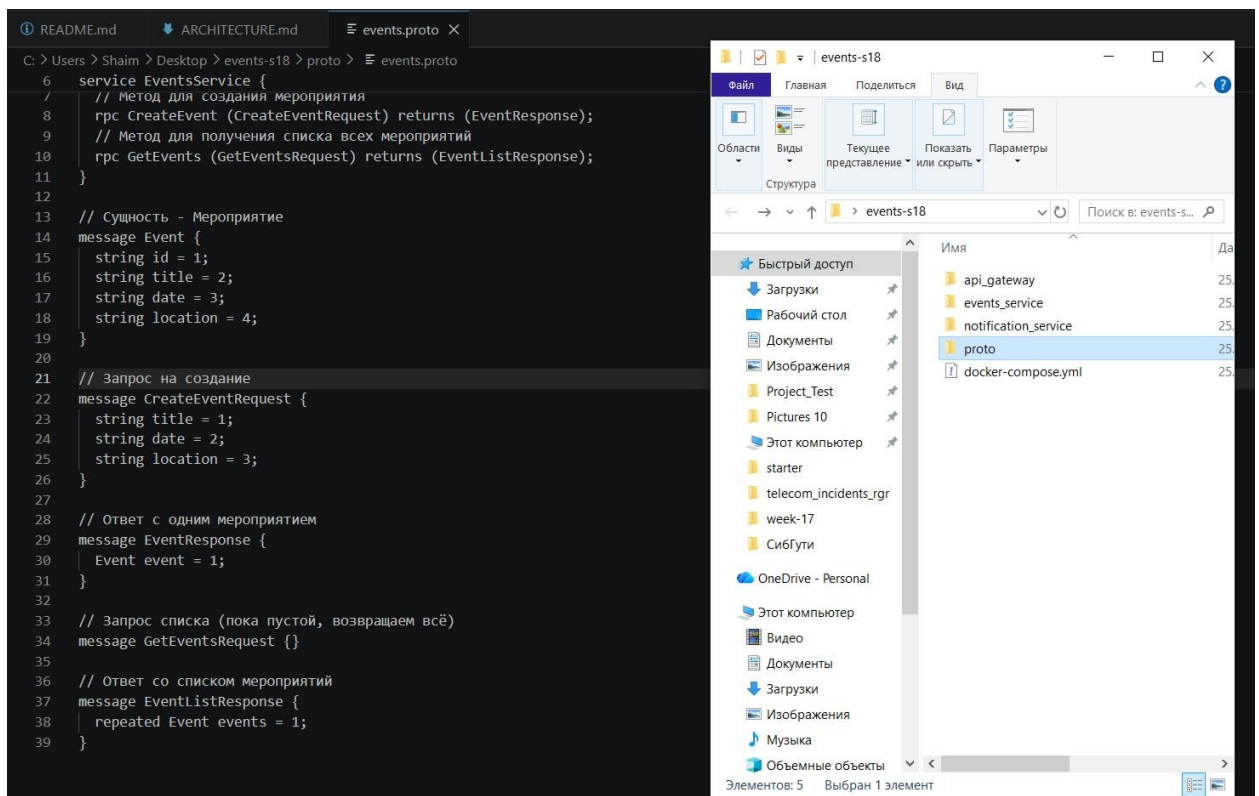


Рисунок 2 - Проектирование структуры микросервисов и разработка протокола взаимодействия gRPC

Листинг 1 — Контракт events.proto

```
syntax = "proto3";
package events.v1;

service EventsService {
  rpc CreateEvent (CreateEventRequest) returns (EventResponse);
  rpc GetEvents (GetEventsRequest) returns (EventListResponse);
}

message Event {
  string id = 1;
  string title = 2;
  string date = 3;
  string location = 4;
}
```

```
message CreateEventRequest {
  string title = 1;
  string date = 2;
  string location = 3;
}
```

```
message EventResponse {
  Event event = 1;
}
```

```
message GetEventsRequest {}

message EventListResponse {
  repeated Event events = 1;
}
```

```

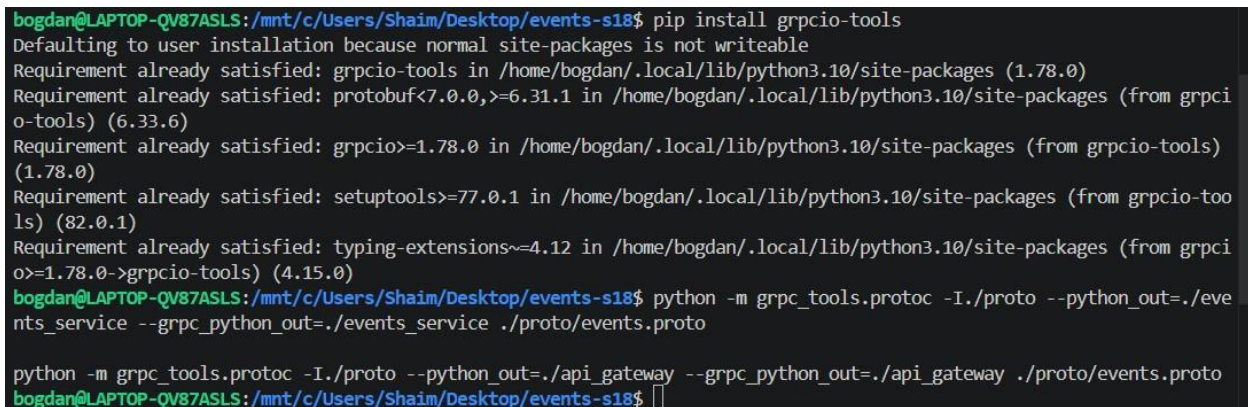
    string location = 4;
}

message CreateEventRequest {
    string title = 1;
    string date = 2;
    string location = 3;
}

message EventResponse { Event event = 1; }
message GetEventsRequest {}
message EventListResponse { repeated Event events = 1; }

```

Для использования данного контракта в Python был применен компилятор `grpc_tools.protoc`, который автоматически сгенерировал классы-заглушки (stubs) для реализации серверной и клиентской логики.



```

bogdan@LAPTOP-QV87ASLS: /mnt/c/Users/Shaim/Desktop/events-s18$ pip install grpcio-tools
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: grpcio-tools in /home/bogdan/.local/lib/python3.10/site-packages (1.78.0)
Requirement already satisfied: protobuf<7.0.0,>=6.31.1 in /home/bogdan/.local/lib/python3.10/site-packages (from grpcio-tools) (6.33.6)
Requirement already satisfied: grpcio>=1.78.0 in /home/bogdan/.local/lib/python3.10/site-packages (from grpcio-tools) (1.78.0)
Requirement already satisfied: setuptools>=77.0.1 in /home/bogdan/.local/lib/python3.10/site-packages (from grpcio-tools) (82.0.1)
Requirement already satisfied: typing-extensions~=4.12 in /home/bogdan/.local/lib/python3.10/site-packages (from grpcio-tools) (4.15.0)
bogdan@LAPTOP-QV87ASLS: /mnt/c/Users/Shaim/Desktop/events-s18$ python -m grpc_tools.protoc -I./proto --python_out=./events_service --grpc_python_out=./events_service ./proto/events.proto

python -m grpc_tools.protoc -I./proto --python_out=./api_gateway --grpc_python_out=./api_gateway ./proto/events.proto
bogdan@LAPTOP-QV87ASLS: /mnt/c/Users/Shaim/Desktop/events-s18$

```

Рисунок 3 - Компиляция Protobuf-контрактов через grpcio-tools

4.2. Разработка микросервиса Events Service

Основной микросервис реализует бизнес-логику системы. Он подключается к реляционной базе данных PostgreSQL с использованием ORM SQLAlchemy. При вызове удаленной процедуры `CreateEvent` сервис выполняет транзакционную запись нового мероприятия в базу данных. Для обеспечения слабой связности архитектуры (Coupling), сервис не вызывает другие микросервисы напрямую. Вместо этого он публикует событие в брокер сообщений RabbitMQ, информируя систему о появлении новых данных. Микросервис прослушивает внутренний порт 8277.

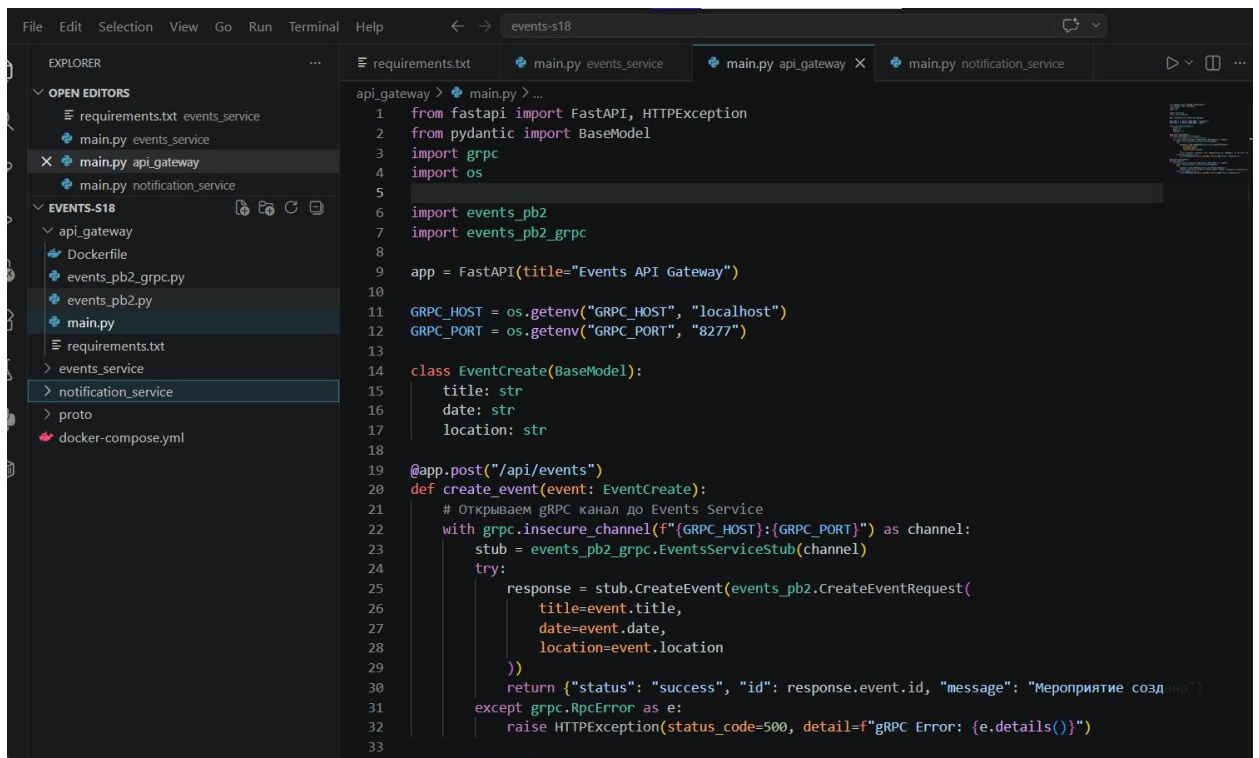


Рисунок 4 - Реализация бизнес-логики микросервиса Events и интеграция с PostgreSQL

4.3. Разработка API Gateway

API Gateway выполняет роль фасада (REST-to-gRPC транслятора). Он разработан на базе высокопроизводительного фреймворка FastAPI. Шлюз принимает HTTP POST и GET запросы от клиентов на эндпоинт /api/events, валидирует JSON-тело с помощью Pydantic и транслирует их в бинарный gRPC-вызов к Events Service.

```

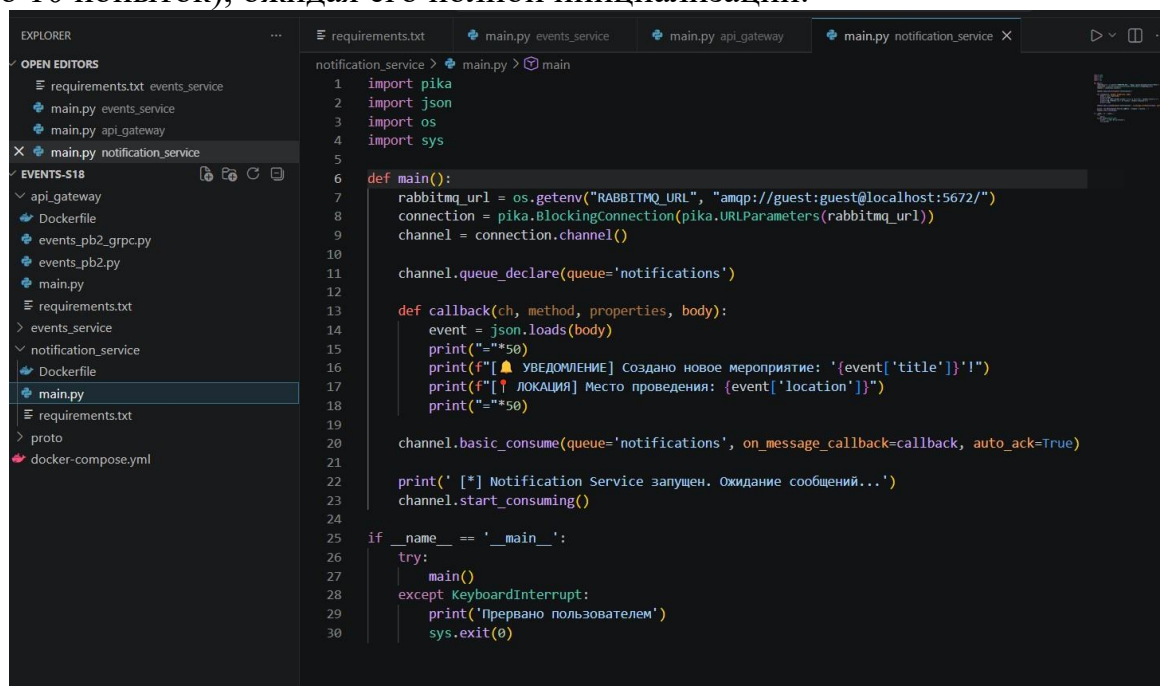
19 @app.post("/api/events")
20 def create_event(event: EventCreate):
21     # Открываем gRPC канал до Events Service
22     with grpc.insecure_channel(f"{GRPC_HOST}:{GRPC_PORT}") as channel:
23         stub = events_pb2_grpc.EventsServiceStub(channel)
24         try:
25             response = stub.CreateEvent(events_pb2.CreateEventRequest(
26                 title=event.title,
27                 date=event.date,
28                 location=event.location
29             ))
30             return {"status": "success", "id": response.event.id, "message": "Мероприятие создано"}
31         except grpc.RpcError as e:
32             raise HTTPException(status_code=500, detail=f"gRPC Error: {e.details()}")
33
34 @app.get("/api/events")
35 def get_events():
36     with grpc.insecure_channel(f"{GRPC_HOST}:{GRPC_PORT}") as channel:
37         stub = events_pb2_grpc.EventsServiceStub(channel)
38         try:
39             response = stub.GetEvents(events_pb2.GetEventsRequest())
40             return [{"id": e.id, "title": e.title, "date": e.date, "location": e.location} for e in response.events]
41         except grpc.RpcError as e:
42             raise HTTPException(status_code=500, detail=f"gRPC Error: {e.details()}")

```

Рисунок 5 - Реализация API Gateway на базе FastAPI и маршрутизация REST-запросов в gRPC

4.4. Асинхронный сервис уведомлений (Notification Service)

Для обработки фоновых задач реализован сервис-подписчик (Consumer). Его задача — непрерывно прослушивать очередь notifications в RabbitMQ. При получении сообщения сервис эмулирует отправку push-уведомления (путем вывода форматированного лога в консоль). В коде реализован механизм отказоустойчивости: сервис циклически пытается подключиться к брокеру (до 10 попыток), ожидая его полной инициализации.



```

1 import pika
2 import json
3 import os
4 import sys
5
6 def main():
7     rabbitmq_url = os.getenv("RABBITMQ_URL", "amqp://guest:guest@localhost:5672/")
8     connection = pika.BlockingConnection(pika.URLParameters(rabbitmq_url))
9     channel = connection.channel()
10
11     channel.queue_declare(queue='notifications')
12
13     def callback(ch, method, properties, body):
14         event = json.loads(body)
15         print(f"УВЕДОМЛЕНИЕ] Создано новое мероприятие: '{event['title']}'!")
16         print(f"↑ ЛОКАЦИЯ] Место проведения: {event['location']}")
17         print(f"")
18
19     channel.basic_consume(queue='notifications', on_message_callback=callback, auto_ack=True)
20
21     print(' [*] Notification Service запущен. Ожидание сообщений...')
22     channel.start_consuming()
23
24
25 if __name__ == '__main__':
26     try:
27         main()
28     except KeyboardInterrupt:
29         print('Прервано пользователем')
30         sys.exit(0)

```

Рисунок 6 - Разработка асинхронного сервиса уведомлений и настройка подписки на брокер сообщений RabbitMQ

5. Инфраструктура и развёртывание (DOCKER)

Для обеспечения удобства разработки и изоляции среды выполнения вся система была контейнеризирована. Для каждого Python-сервиса был написан Dockerfile, базирующийся на легковесном образе python:3.10-slim.

Оркестрация компонентов (3 микросервиса, PostgreSQL и RabbitMQ) выполняется с помощью утилиты docker-compose.

В ходе развёртывания была успешно решена классическая проблема инфраструктуры — конфликт сетевых портов базы данных (порт 5432 был занят хост-машиной). Проблема была устранена путем изоляции базы данных: проброс портов наружу был отключен, и всё взаимодействие между Events Service и PostgreSQL было переведено в закрытую виртуальную сеть Docker (events-net).

Листинг 2 — Оркестрация среды в docker-compose.yml (фрагмент)

```
services:
  rabbitmq:
    image: rabbitmq:3-management
    networks:
      - events-net

  postgres_db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: events_db
    networks:
      - events-net

  events_service:
    build: ./events_service
    ports:
      - "8277:8277"
    depends_on:
      - postgres_db
      - rabbitmq
    networks:
      - events-net

# Конфигурация API Gateway и Notification Service...
```

6. Тестирование программно - аппаратного комплекса

Для верификации работы системы был произведен комплексный запуск командой docker-compose up. Система успешно инициализировала БД, подняла брокер сообщений и запустила все зависимые микросервисы.

```
Feature `transient_nonexcl_queues` is deprecated.
rabbitmq-s18 | 2026-05-25 08:28:51.316755+00:00 [warning] <0.869.0> By default, this feature can still be used for now.
rabbitmq-s18 | 2026-05-25 08:28:51.316755+00:00 [warning] <0.869.0> Its use will not be permitted by default in a future minor RabbitMQ version and the feature will be removed from a future major RabbitMQ version; actual versions to be determined.
rabbitmq-s18 | 2026-05-25 08:28:51.316755+00:00 [warning] <0.869.0> To continue using this feature when it is not permitted by default, set the following parameter in your configuration:
rabbitmq-s18 | 2026-05-25 08:28:51.316755+00:00 [warning] <0.869.0> "deprecated_features.permit.transient_nonexcl_queues = true"
rabbitmq-s18 | 2026-05-25 08:28:51.316755+00:00 [warning] <0.869.0> To test RabbitMQ as if the feature was removed, set this in your configuration:
rabbitmq-s18 | 2026-05-25 08:28:51.316755+00:00 [warning] <0.869.0> "deprecated_features.permit.transient_nonexcl_queues = false"
postgres-s18 | 2026-05-25 08:33:37.920 UTC [63] LOG: checkpoint starting: time
postgres-s18 | 2026-05-25 08:33:42.066 UTC [63] LOG: checkpoint complete: wrote 44 buffers (0.3%); 0 WAL file(s) added, 0 removed, 0 recycled; write=4.116 s, sync=0.018 s, total=4.146 s; sync files=12, longest=0.015 s, average=0.002 s; distance=252 kB, estimate=252 kB
api-gateway-s18 | INFO: 172.24.0.1:52122 - "GET /docs HTTP/1.1" 200 OK
api-gateway-s18 | INFO: 172.24.0.1:52122 - "GET /openapi.json HTTP/1.1" 200 OK
```

Рисунок 7 - Всё успешно работает

Функциональное тестирование проводилось через автогенерируемую документацию Swagger UI. Был сформирован валидный HTTP POST запрос к API Gateway с полезной нагрузкой:

```
{"title": "Финальный экзамен", "date": "2026-05-28", "location": "Аудитория 317"}.
```

API Gateway успешно преобразовал запрос, передал его по gRPC, база данных сохранила запись, а асинхронный воркер RabbitMQ вывел уведомление в консоль оркестратора. Ответ сервера со статусом 200 ОК подтверждает полную работоспособность разработанной микросервисной архитектуры.

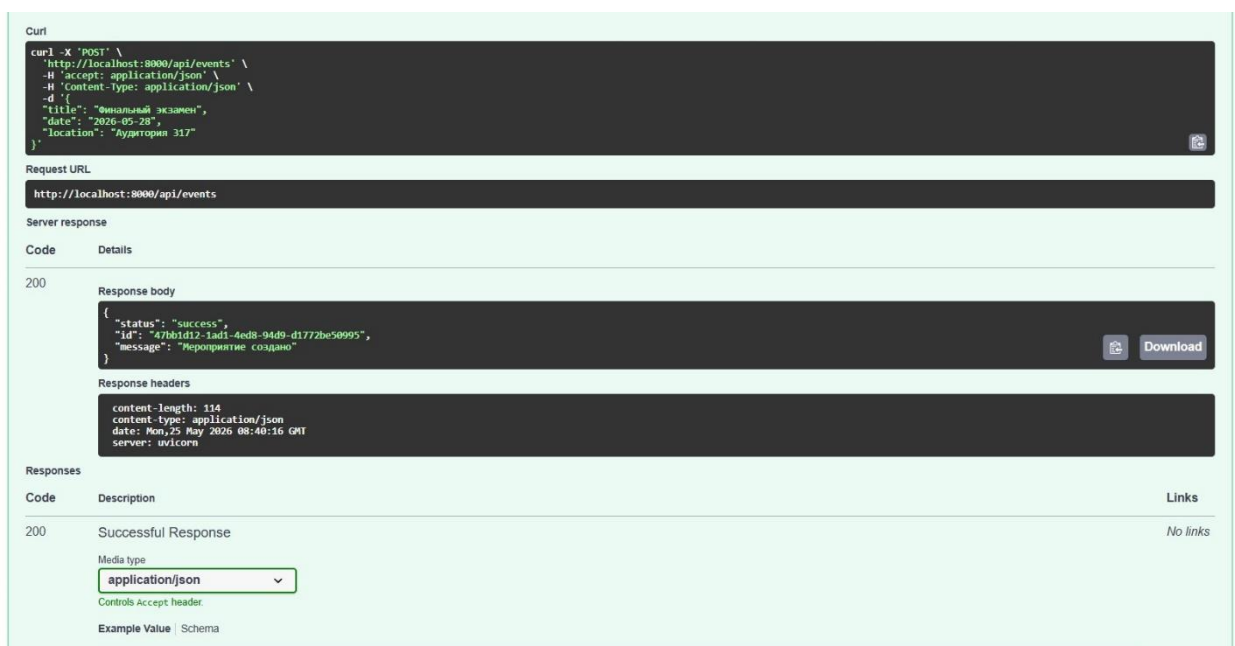


Рисунок 8 - Ответ получен успешно

7. ЗАКЛЮЧЕНИЕ

В ходе выполнения расчетно-графической работы был успешно спроектирован, реализован и протестирован распределенный программно-аппаратный комплекс управления мероприятиями на базе современной микросервисной архитектуры.

В процессе выполнения работы были закреплены следующие ключевые инженерные компетенции:

1. Проектирование API и сетевых контрактов: На практике реализована гибридная схема взаимодействия компонентов: синхронный интерфейс REST HTTP на внешнем контуре для обеспечения доступности клиентам и высокоскоростной типизированный протокол gRPC для межсервисного обмена.
2. Обеспечение слабой связности (Low Coupling): С помощью интеграции брокера сообщений RabbitMQ и реализации асинхронного сервиса уведомлений достигнута изоляция бизнес-компонентов. Система защищена от каскадных сбоев, что гарантирует непрерывность выполнения ключевых операций.
3. Оркестрация и DevOps-практики: Весь комплекс упакован в изолированные Docker-контейнеры и разворачивается одной командой через Docker Compose, что решает проблему консистентности окружения при переносе кода между машинами разработчиков. Были отработаны практические навыки разрешения сетевых инфраструктурных конфликтов на уровне портов ОС.

Разработанная система полностью отвечает критериям надежности, масштабируемости, удобства разработки и наблюдаемости, предъявляемым к современному распределенному программному обеспечению.